Universidad Tecnológica Nacional – Facultad Regional Mendoza

Tecnicatura Universitaria en Programación – Modalidad a distancia

Trabajo Práctico Integrador

Food Store – Sistema de gestión de pedidos de comida (consola)

Materia: Programación II

Estudiante: Erika Janet Oviedo

Legajo: 54286

Fecha de entrega: Noviembre de 2026

1. Introducción.....2
1.1 Objetivos.....2
1.2 Alcance del sistema.....2
2. Marco teórico.....3
2.1 Python y type hints.....3
2.2 Clases, objetos y constructores.....3
2.3 Clase abstracta.....3
2.4 Interfaz.....4
2.5 Herencia y sobrescritura.....4
2.6 Polimorfismo.....4
2.7 Encapsulamiento.....4
2.8 Composición y agregación.....5
2.9 Excepciones.....5
2.10 Colecciones.....6
3. Decisiones técnicas y arquitectura.....7
3.1 Cómo se abordó la solución.....7
3.2 Organización de paquetes.....7
3.3 Modelo de clases.....8
3.4 Decisiones de diseño.....9
3.5 Reglas de negocio y dónde se implementan.....10
3.6 Capturas de pantalla del sistema funcionando.....11
3.7 Pruebas.....19
4. Dificultades y soluciones.....20
5. Conclusiones.....22
6. Bibliografía y webgrafía.....23
7. Enlaces.....23

1. Introducción

Este informe documenta el desarrollo de **Food Store**, un sistema de consola para la gestión de un negocio de comidas: categorías, productos, usuarios y pedidos, con inicio de sesión por rol. El trabajo se realizó para el Trabajo Práctico Integrador de Programación II y se implementó en Python 3.12 aplicando Programación Orientada a Objetos (POO), siguiendo el diagrama UML y el backlog de historias de usuario provistos en el enunciado.

Toda la información vive en memoria mientras el programa se ejecuta, guardada en colecciones; al cerrarlo los datos se pierden, tal como pide la consigna. El código está organizado en tres capas (entidades, servicios y consola) con dependencia en un solo sentido, y se usa únicamente la biblioteca estándar de Python.

1.1 Objetivos

- Modelar las clases y relaciones del UML: herencia, asociación 1..N y N..1, agregación y composición.
- Aplicar clase abstracta, interfaz, sobrescritura de métodos, encapsulamiento y polimorfismo.
- Gestionar las entidades con colecciones y proteger su estado con validaciones en los setters.
- Definir y capturar excepciones propias para que ningún dato mal ingresado cierre el programa.
- Resolver el menú de cada rol con polimorfismo, sin condicionales por rol repartidos en el código.

1.2 Alcance del sistema

Aspecto	Definición
Modalidad	Aplicación de consola con menús
Lenguaje	Python 3.12 o superior
Almacenamiento	En memoria (listas de objetos)
Capas	Entidades (modelo) + Servicios (lógica) + Consola (menús)
Bibliotecas	Solo biblioteca estándar: abc, enum, datetime, getpass
Roles	ADMIN (alta, baja, modificación y consulta) y USUARIO (solo consulta)
Tamaño	35 archivos .py, unas 2100 líneas, con type hints verificados con mypy --strict

2. Marco teórico

Esta sección resume los conceptos del paradigma que se aplicaron en el proyecto y señala, en cada caso, dónde aparecen en el código.

2.1 Python y type hints

Python es un lenguaje de programación de alto nivel, interpretado y multiparadigma (permite programación imperativa, orientada a objetos y funcional). Es de tipado dinámico: una variable puede referenciar objetos de distinto tipo durante la ejecución. Su sintaxis legible y su biblioteca estándar extensa lo hacen apropiado para aprender POO.

Los **type hints** (PEP 484) son anotaciones opcionales que indican el tipo esperado de atributos, parámetros y valores de retorno, por ejemplo `def precio(self) -> float`. Python no los verifica al ejecutar, pero documentan el contrato de cada clase y método y permiten que herramientas como *mypy* detecten errores antes de correr el programa. Desde Python 3.10 se puede escribir `DetallePedido | None` para indicar un valor opcional. En este proyecto todo atributo, parámetro y retorno está anotado, y el código completo pasa `mypy --strict` sin errores.

2.2 Clases, objetos y constructores

Una **clase** es un molde que define atributos (estado) y métodos (comportamiento); un **objeto** es una instancia concreta de ella. El constructor `__init__` inicializa el estado del objeto. Cada entidad del UML (Categoria, Producto, Usuario, Pedido, DetallePedido) es una clase cuyo constructor recibe y valida sus datos.

2.3 Clase abstracta

Una clase abstracta no puede instanciarse: define un contrato y código común para sus subclases. En Python se declara heredando de `abc.ABC` y marcando métodos con `@abstractmethod`; la subclase debe implementarlos para poder instanciarse (PEP 3119).

En el proyecto: `Base` es la superclase abstracta de todas las entidades. Aporta el *id* automático, la baja lógica (*eliminado*), la fecha de creación (*creado_en*) y `__eq__`. Como `ABC` por sí sola no impide instanciar una clase sin métodos abstractos, `Base` declara `__str__` como abstracto, lo que además obliga a cada entidad a mostrarse con una línea útil. `Menu` también es abstracta.

```
class Base(ABC):
    __contadores: dict[type, int] = {}

    def __init__(self) -> None:
        clase: type = type(self)
        Base.__contadores[clase] = Base.__contadores.get(clase, 0) + 1
        self.__id: int = Base.__contadores[clase]
        self.__eliminado: bool = False
        self.__creado_en: datetime = datetime.now()
```

2.4 Interfaz

Una interfaz define *qué* debe saber hacer un objeto sin decir *cómo*. Python no tiene la palabra reservada *interface*; se modela con una clase abstracta que solo contiene métodos abstractos y ningún atributo. Cualquier clase que la implemente se compromete a ofrecer esos métodos.

En el proyecto: `Calculable` declara un único método abstracto, `calcular_total() -> None`, y `Pedido` la implementa (`class Pedido(Base, Calculable)`).

```
class Calculable(ABC):
    @abstractmethod
    def calcular_total(self) -> None:
        ...
```

2.5 Herencia y sobrescritura

La **herencia** permite que una clase reutilice atributos y métodos de otra (relación «es un»). La **sobrescritura** redefine en la subclase un método heredado. Python admite herencia múltiple y resuelve el orden de búsqueda de métodos con el MRO.

En el proyecto: las cinco entidades heredan de `Base` y llaman a `super().__init__()`. Cada una sobrescribe `__str__` con una línea para los listados, y `Base` sobrescribe `__eq__` (y define `__hash__` de forma coherente) para comparar objetos por tipo e id. `Pedido` usa herencia múltiple: hereda de `Base` e implementa `Calculable`.

2.6 Polimorfismo

El **polimorfismo** permite enviar el mismo mensaje a objetos de distintas clases y que cada uno responda a su manera, sin que quien lo invoca conozca la clase concreta.

En el proyecto: el menú de cada rol se modela con la clase abstracta `Menu` (método abstracto `mostrar() -> bool`) y una subclase por rol: `MenuAdministrador` y `MenuCliente`. El rol se consulta en un único lugar, la función `crear_menu`; después `main()` solo llama a `menu.mostrar()`. Agregar un rol nuevo implica agregar una subclase, sin tocar las existentes.

```
def crear_menu(usuario, categorias, productos, usuarios, pedidos) -> Menu:
    if usuario.rol == Rol.ADMIN:
        return MenuAdministrador(usuario, categorias, productos, usuarios, pedidos)
    return MenuCliente(usuario, categorias, productos, pedidos)

# en main():
menu = crear_menu(sesion, categorias, productos, usuarios, pedidos)
if not menu.mostrar():  # False = salir del programa
    break
```

2.7 Encapsulamiento

El **encapsulamiento** protege el estado interno de un objeto: los atributos son privados y solo se acceden mediante métodos o *properties* que validan los datos. En Python, el prefijo de doble guion bajo (`self.__precio`) activa el *name mangling* y evita el acceso accidental desde afuera; `@property` expone la lectura y `@atributo.setter` permite validar cada modificación.

En el proyecto: todos los atributos son privados. Solo tienen setter los que el sistema permite modificar, y cada setter valida y lanza una excepción propia. La contraseña es de solo escritura: no existe forma de leerla, y `Usuario` ofrece `verificar_contrasenia()` para que nunca salga del objeto. La lista de detalles de un pedido se expone como tupla, así no puede modificarse desde afuera.

```
@property
def precio(self) -> float:
    return self.__precio

@precio.setter
def precio(self, valor: float) -> None:
    if isinstance(valor, bool) or not isinstance(valor, (int, float)):
        raise PrecioInvalidoError("El precio debe ser un número")
    if valor < 0:
        raise PrecioInvalidoError("El precio no puede ser negativo")
    self.__precio: float = float(valor)
```

2.8 Composición y agregación

Ambas son asociaciones «tiene un». En la **composición** (rombo relleno) la parte no existe sin el todo: comparten ciclo de vida. En la **agregación** (rombo vacío) las partes tienen vida propia y pueden pertenecer a otro agregado.

- **Composición Pedido – DetallePedido (1..N):** solo `Pedido` crea detalles (con `agregar_detalle`), los guarda en una lista privada y los da de baja junto con el pedido: «el detalle nace y muere con su pedido».
- **Agregación Categoria – Producto (1..N):** la categoría conoce sus productos y puede contarlos, pero un producto es una entidad independiente que puede cambiar de categoría. El setter `Producto.categoria` mantiene ambos lados sincronizados.
- **Asociación Usuario – Pedido (1..N)** y **DetallePedido – Producto (N..1):** el pedido referencia a su usuario y cada detalle referencia a su producto.

2.9 Excepciones

Una **excepción** es un objeto que representa una situación anormal y que interrumpe el flujo normal hasta que alguien la captura con `try / except`. Lanzar excepciones propias (`raise`) permite que las capas inferiores informen errores de negocio con significado, y que la consola los capture para mostrar un mensaje y volver al menú sin cerrarse.

En el proyecto: todas heredan de `FoodStoreError`. La consola captura `FoodStoreError` para los errores de negocio y `ValueError` para las conversiones fallidas de `int()` y `float()`.

Excepción	Cuándo se lanza
`EntidadNoEncontradaError`	Id inexistente o entidad eliminada.
`DatoInvalidoError`	Texto en blanco, mail sin formato válido, tipo incorrecto.

Excepción	Cuándo se lanza
`PrecioInvalidoError` / `StockInvalidoError` / `CantidadInvalidaError`	Subclases de DatoInvalidoError: precio o stock negativos, cantidad menor o igual a 0.
`StockInsuficienteError`	La cantidad pedida supera el stock del producto.
`DuplicadoError`	Mail de usuario o nombre de categoría repetido.
`OperacionNoPermitidaError`	Transición de estado inválida, categoría con productos activos, administrador que se elimina a sí mismo, falta de permisos.
`CredencialesInvalidasError`	Mail o contraseña incorrectos, o usuario eliminado.

2.10 Colecciones

Las colecciones de Python almacenan y organizan objetos: *list* (ordenada y modificable), *tuple* (ordenada e inmutable), *dict* (pares clave-valor) y *set* (sin repetidos). El proyecto las usa así:

- **list:** cada servicio guarda en una lista privada los objetos de su entidad (con baja lógica, el objeto sigue en la lista con *eliminado = True*).
- **tuple:** `Pedido.detalles` y `Categoria.productos` devuelven tuplas para evitar modificaciones externas.
- **dict:** `TRANSICIONES_VALIDAS` define qué estados puede alcanzar cada estado; `MenuAdministrador` asocia número de opción con su gestión.
- **set:** conjuntos de opciones válidas al leer el menú.

3. Decisiones técnicas y arquitectura

3.1 Cómo se abordó la solución

El desarrollo siguió el orden de dependencias del proyecto, de adentro hacia afuera, y cada etapa se cerró antes de pasar a la siguiente:

- **Enumerados y excepciones** (*Rol, Estado, FormaPago* y la jerarquía de *FoodStoreError*), porque el resto del código depende de ellos.
- **Entidades** según el UML: *Base*, *Calculable*, *Categoria*, *Producto*, *Usuario*, *DetallePedido* y *Pedido*, con atributos privados y setters que validan.
- **Servicios:** una colección y las reglas de negocio por entidad, más *permisos* para verificar el rol.
- **Consola:** lectura validada, vistas, menú abstracto, un menú por rol y una gestión por entidad.
- **Pruebas** (*pruebas.py*) del modelo y los servicios sin menú, y verificación de type hints con *mypy --strict*.
- **Ejecución manual** de todos los flujos de las historias de usuario para obtener las capturas de pantalla de este informe.

3.2 Organización de paquetes

```
food_store/
├── main.py              # crea los servicios y arranca el ciclo de sesión
├── datos_iniciales.py   # administrador inicial y datos de ejemplo (vía servicios)
├── pruebas.py           # pruebas del modelo y servicios, sin menú
├── entidades/               # Base (abstracta), Calculable (interfaz), Categoria,
Producto, Usuario, Pedido, DetallePedido
├── enums/               # Rol, Estado, FormaPago
├── excepciones/         # FoodStoreError y sus subclases
├── servicios/           # un servicio por entidad + permisos
└── consola/                  # Menu (abstracta), MenuAdministrador, MenuCliente,
gestiones y lectura validada
```

La dependencia va siempre en un solo sentido: **consola** → **servicios** → **entidades**. Las responsabilidades de cada capa son:

Capa	Hace	No hace
entidades	Guardan su estado, lo validan en los setters y se muestran con `__str__`.	No usan `input()` ni `print()`; no conocen las colecciones.
servicios	Tienen la colección de su entidad, buscan por id y aplican reglas que involucran varios objetos (mail único, nombre de categoría repetido, stock, permisos).	No leen del teclado ni imprimen.
consola	Piden datos, arman los objetos a través de los servicios, capturan excepciones y muestran mensajes.	No contienen reglas de negocio.

Capa	Hace	No hace
main.py	Crea los servicios, carga los datos iniciales y arranca el ciclo de sesión.	No contiene reglas de negocio.

3.3 Modelo de clases

La Figura 1 muestra el diagrama de clases implementado. Coincide con el UML del enunciado: *Base* es la superclase abstracta de las cinco entidades, *Pedido* implementa *Calculable*, hay composición Pedido–DetallePedido, agregación Categoria–Producto y los tres enumerados.

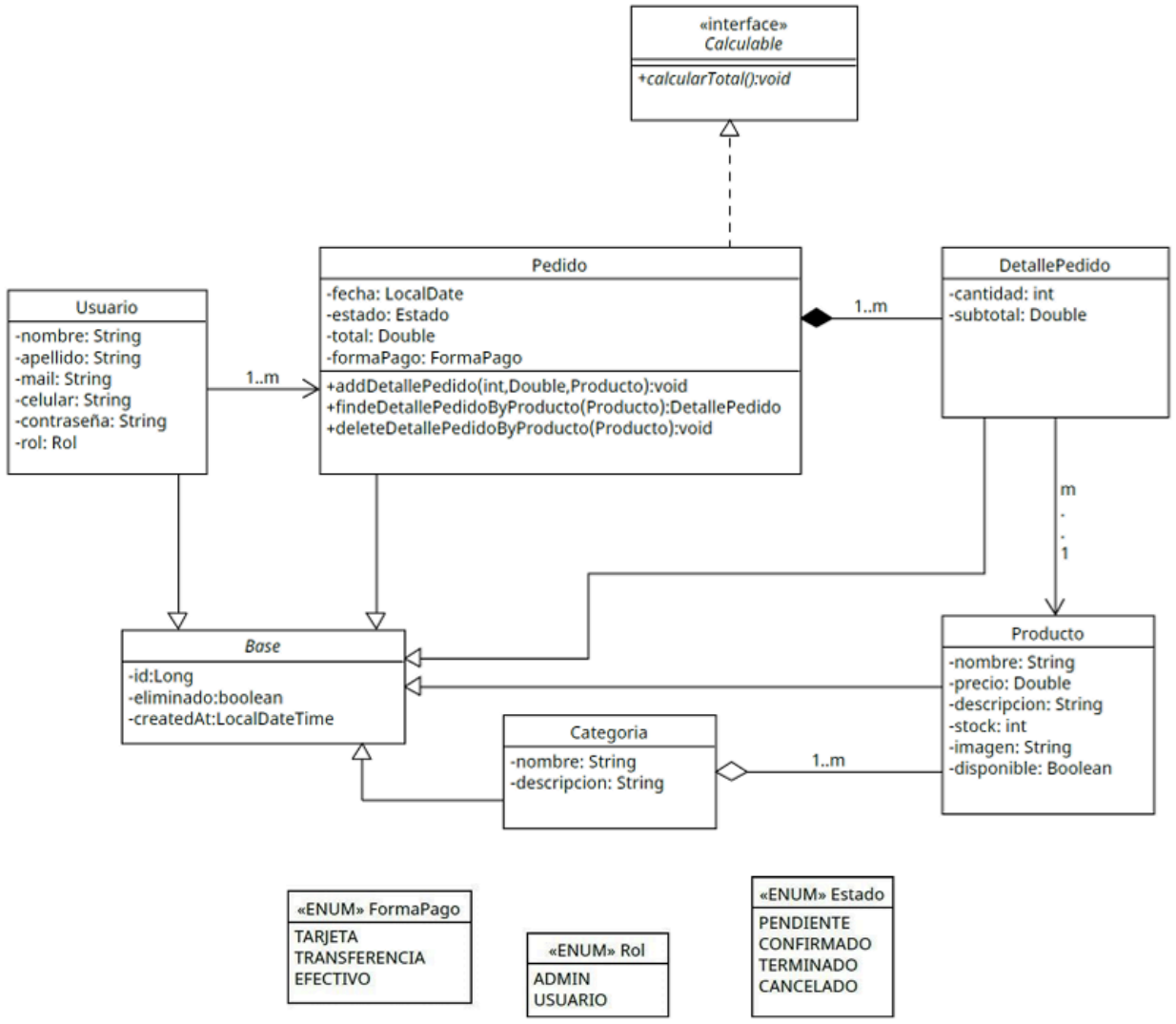


Figura 1 – Diagrama de clases de Food Store

Servicio	Responsabilidad	Operaciones principales
CategoriaServicio	Colección de categorías; nombre único; baja con control de productos activos.	`crear, obtener, listar, editar, eliminar`
ProductoServicio	Colección de productos; validación de categoría; vista del cliente (disponibles y con stock).	`crear, obtener, listar, listar_disponibles, editar, eliminar`
UsuarioServicio	Colección de usuarios; mail único; autenticación; administrador inicial.	`crear, autenticar, obtener, listar, editar, eliminar, registrar_administrador_inicial`
PedidoServicio	Colección de pedidos; creación en pasos; descuento y devolución de stock; estados.	`crear_borrador, agregar_detalle, registrar, listar, listar_propios, obtener_propio, actualizar, eliminar`

3.4 Decisiones de diseño

1. **Id por clase.** El enunciado permite un contador compartido o uno por clase. Se eligió un contador por clase: para quien opera la consola es más natural escribir «categoría 2» o «producto 5» que ids salteados entre entidades. Como los ids ya no son únicos entre tipos distintos, `__eq__` compara tipo e id. Los ids no se reutilizan nunca (igual que una secuencia de base de datos): un pedido armado y descartado por falta de stock consume su número.
2. **Ids correlativos ante errores de validación.** Los constructores validan sus datos antes de llamar a `super().__init__()`; así, un alta rechazada por precio negativo o mail inválido no consume un id.
3. **Agregación bidireccional Categoria–Producto.** Categoria guarda sus productos y expone `cantidad_productos_activos()`, que usa el servicio para aplicar la regla «no se elimina una categoría con productos activos» e informar cuántos tiene.
4. **Precio unitario congelado en el detalle.** `DetallePedido` guarda el precio del producto en el momento de agregarlo. Así el subtotal (cantidad × precio) no depende de un dato que escribe el operador y un cambio posterior de precio no altera pedidos existentes.
5. **Creación de pedido en tres pasos.** *crear_borrador* arma el pedido en memoria, *agregar_detalle* valida cantidad y stock sin descontar nada y *registrar* calcula el total con `calcular_total()`, descuenta el stock y recién ahí agrega el pedido a la colección. Si falla cualquier detalle, el borrador se descarta: el pedido no se guarda y ningún stock cambia (historia HU-PED-02).
6. **Transiciones de estado dentro de Pedido.** El setter de `estado` consulta el diccionario `TRANSICIONES_VALIDAS`, de modo que ninguna parte del código puede saltarse la regla. El servicio devuelve el stock cuando el nuevo estado es CANCELADO.
7. **Permisos en los servicios.** Además de ocultar opciones con el menú por rol, cada operación que modifica datos recibe el usuario que la ejecuta y llama a `exigir_admin()`. La regla «solo un ADMIN crea, edita o elimina» no depende solo de la interfaz y se puede probar sin menú.

8. **Ediciones atómicas.** Al editar varios campos de una vez, si uno es inválido se restauran los valores anteriores y se propaga el error: nunca queda un objeto modificado a medias.
9. **Bajas lógicas.** Eliminar marca *eliminado = True*; el objeto sigue en la colección. Los listados filtran los eliminados, los pedidos existentes siguen mostrando productos dados de baja y el mail de un usuario eliminado sigue reservado.
10. **Administrador inicial.** `registrar_administrador_inicial` permite crear el primer administrador (sin él nadie podría ingresar) y se rechaza si ya existe uno activo, para que no sea una puerta trasera.
11. **Menú por rol y sesión local.** `crear_menu` es el único lugar donde se pregunta el rol. La sesión es una variable local de `main()` que se pasa al menú; no hay variables globales.
12. **Lectura validada y robustez.** El módulo *entrada* repite cada pregunta hasta recibir un dato válido, las opciones de enumerados se eligen por número y toda acción de menú se ejecuta con *ejecutar_seguro*, que informa el error y vuelve al menú. Si la entrada se cierra (Ctrl+D) o se interrumpe (Ctrl+C), *main()* termina ordenadamente.
13. **Contraseñas.** Se guardan en memoria como texto, tal como plantea el enunciado, y se ocultan al escribirlas con `getpass`. En un sistema real se almacenaría un hash con sal; queda como mejora futura.

3.5 Reglas de negocio y dónde se implementan

Regla de negocio	Implementación
Precio y stock no pueden ser negativos	Producto: setters → PrecioInvalidoError / StockInvalidoError
Pedido sin usuario o con usuario eliminado	Pedido.__init__ → DatoInvalidoError / OperacionNoPermitidaError
Cantidad > 0 y no superior al stock	Pedido.agregar_detalle → CantidadInvalidaError / StockInsuficienteError
Un solo detalle por producto (se suma la cantidad)	Pedido.agregar_detalle y buscar_detalle_por_producto
Mail de usuario único	UsuarioServicio (recorre la colección, sin distinguir mayúsculas) → DuplicadoError
Nombre de categoría único sin distinguir mayúsculas	CategoriaServicio → DuplicadoError
No eliminar categoría con productos activos	CategoriaServicio.eliminar → OperacionNoPermitidaError (informa cuántos tiene)
Transiciones de estado válidas	Pedido.estado + TRANSICIONES_VALIDAS → OperacionNoPermitidaError
Devolver stock al cancelar o eliminar un pedido no terminado	PedidoServicio.actualizar y eliminar

Regla de negocio	Implementación
Toda baja es lógica	Base.eliminado; los servicios filtran los eliminados
Solo ADMIN crea, edita o elimina	servicios.permisos.exigir_admin + MenuCliente sin opciones de modificación
Un usuario eliminado no inicia sesión	UsuarioServicio.autenticar → CredencialesInvalidasError
Un administrador no se elimina a sí mismo	UsuarioServicio.eliminar → OperacionNoPermitidaError
Mensaje de login idéntico en los tres casos de error	autenticar siempre lanza «Mail o contraseña incorrectos»; 3 intentos y termina

3.6 Capturas de pantalla del sistema funcionando

Las siguientes capturas corresponden a ejecuciones reales de `python main.py` con los datos de ejemplo que se cargan al iniciar (administrador *admin@foodstore.com* y cliente *juan@foodstore.com*).

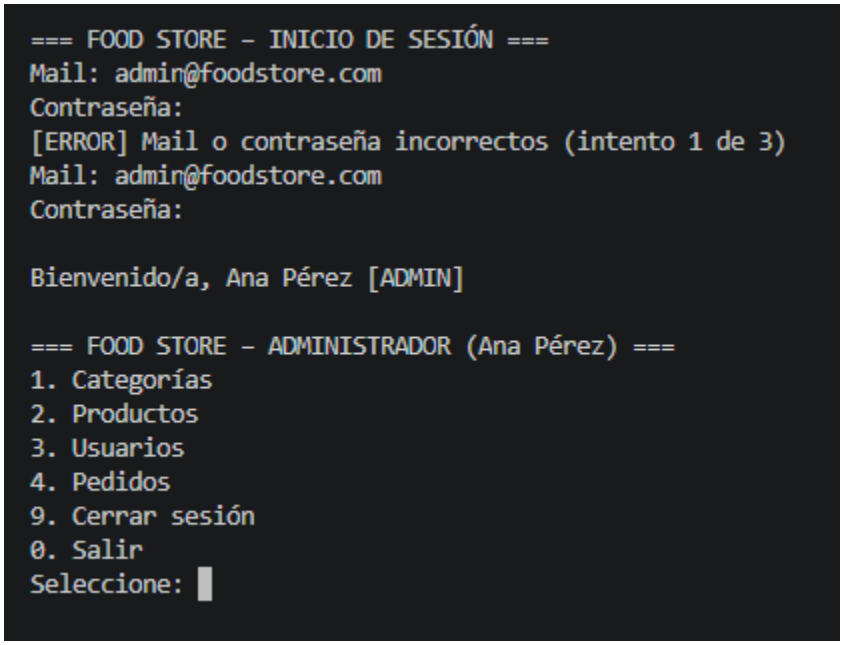


Figura 2 – `01_login_admin`

Inicio de sesión: un intento fallido (mensaje genérico), ingreso correcto y menú del administrador.

```
=== FOOD STORE – ADMINISTRADOR (Ana Pérez) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
9. Cerrar sesión
0. Salir
Seleccione: 1

--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Hamburguesas
Descripción: Hamburguesas
[ERROR] Ya existe una categoría llamada 'Hamburguesas'

--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Sandwiches
Descripción: Sandwiches de miga
[OK] Categoría creada con id 5.

--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 4
  [1] Hamburguesas - Hamburguesas artesanales
  [2] Pizzas - Pizzas al horno de piedra
  [3] Bebidas - Gaseosas y aguas
  [4] Postres - Dulces caseros
  [5] Sandwiches - Sandwiches de miga
Id de la categoría a eliminar: 1
¿Eliminar la categoría 'Hamburguesas'? (S/N): S
[ERROR] No se puede eliminar la categoría 'Hamburguesas': tiene 2 producto(s) activo(s)
```

Figura 3 – `02_categorias`

Categorías: nombre repetido rechazado, alta correcta y baja rechazada por tener productos activos.

```
--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Empanada
Descripción: Carne cortada a cuchillo
Precio: -50
Stock: 2
Imagen (ruta o URL):
¿Disponible? (S/N): S
  [1] Hamburguesas - Hamburguesas artesanales
  [2] Pizzas - Pizzas al horno de piedra
  [3] Bebidas - Gaseosas y aguas
  [4] Postres - Dulces caseros
  [5] Sandwiches - Sandwiches de miga
Id de la categoría: 2
[ERROR] El precio no puede ser negativo

--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Jugo de naranja
Descripción: Naranja exprimida
Precio: 2000
Stock: 23
Imagen (ruta o URL):
¿Disponible? (S/N): s
  [1] Hamburguesas - Hamburguesas artesanales
  [2] Pizzas - Pizzas al horno de piedra
  [3] Bebidas - Gaseosas y aguas
  [4] Postres - Dulces caseros
  [5] Sandwiches - Sandwiches de miga
Id de la categoría: 3
[OK] Producto creado con id 9.
```

Figura 4 – `03_validacion_producto`

Validaciones al crear un producto: número mal ingresado (se repite la pregunta) y precio negativo (PrecioInvalidoError).

```
--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Cambiar estado y/o forma de pago
6. Eliminar
0. Volver
Seleccione: 4
  [1] Ana Pérez - admin@foodstore.com - ADMIN
  [2] Juan Gómez - juan@foodstore.com - USUARIO
Id del usuario: 2
Forma de pago:
  1. TARJETA
  2. TRANSFERENCIA
  3. EFECTIVO
Seleccione: 1
  [1] Hamburguesa clásica - $4500.00 - stock 18 - Hamburguesas (disponible)
  [2] Hamburguesa doble cheddar - $6200.00 - stock 15 - Hamburguesas (disponible)
  [3] Pizza muzzarella - $7000.00 - stock 10 - Pizzas (disponible)
  [4] Pizza napolitana - $8200.00 - stock 3 - Pizzas (disponible)
  [5] Gaseosa 500 ml - $1800.00 - stock 39 - Bebidas (disponible)
  [6] Agua mineral - $1200.00 - stock 50 - Bebidas (disponible)
  [7] Flan casero - $2500.00 - stock 12 - Postres (disponible)
  [8] Brownie con helado - $3800.00 - stock 8 - Postres (no disponible)
  [9] Jugo de naranja - $2000.00 - stock 23 - Bebidas (disponible)
Id del producto: 3
Cantidad: 2
[OK] Detalle agregado. Total parcial: $14000.00
¿Agregar otro producto? (S/N): s
  [1] Hamburguesa clásica - $4500.00 - stock 18 - Hamburguesas (disponible)
  [2] Hamburguesa doble cheddar - $6200.00 - stock 15 - Hamburguesas (disponible)
  [3] Pizza muzzarella - $7000.00 - stock 10 - Pizzas (disponible)
  [4] Pizza napolitana - $8200.00 - stock 3 - Pizzas (disponible)
  [5] Gaseosa 500 ml - $1800.00 - stock 39 - Bebidas (disponible)
  [6] Agua mineral - $1200.00 - stock 50 - Bebidas (disponible)
  [7] Flan casero - $2500.00 - stock 12 - Postres (disponible)
  [8] Brownie con helado - $3800.00 - stock 8 - Postres (no disponible)
  [9] Jugo de naranja - $2000.00 - stock 23 - Bebidas (disponible)
Id del producto: 5
Cantidad: 3
[OK] Detalle agregado. Total parcial: $19400.00
¿Agregar otro producto? (S/N): n
[OK] Pedido creado con id 2. Total: $19400.00
```

Figura 5 – `04_pedido_ok`

Alta de un pedido con dos detalles: el total se calcula con calcular_total().

```
Id del usuario: 2
Forma de pago:
  1. TARJETA
  2. TRANSFERENCIA
  3. EFECTIVO
Seleccione: 1
  [1] Hamburguesa clásica - $4500.00 - stock 18 - Hamburguesas (disponible)
  [2] Hamburguesa doble cheddar - $6200.00 - stock 15 - Hamburguesas (disponible)
  [3] Pizza muzzarella - $7000.00 - stock 8 - Pizzas (disponible)
  [4] Pizza napolitana - $8200.00 - stock 3 - Pizzas (disponible)
  [5] Gaseosa 500 ml - $1800.00 - stock 36 - Bebidas (disponible)
  [6] Agua mineral - $1200.00 - stock 50 - Bebidas (disponible)
  [7] Flan casero - $2500.00 - stock 12 - Postres (disponible)
  [8] Brownie con helado - $3800.00 - stock 8 - Postres (no disponible)
  [9] Jugo de naranja - $2000.00 - stock 23 - Bebidas (disponible)
Id del producto: 4
Cantidad: 6
[ERROR] Stock insuficiente de 'Pizza napolitana': pedido 6, disponible 3
Creación cancelada: el pedido no se guardó y no se modificó ningún stock.
```

Figura 6 – `05_pedido_sin_stock`

Pedido cancelado por falta de stock: no se guarda el pedido ni se modifica ningún stock.

```
Seleccione: 5
  [1] Juan Gómez | PENDIENTE | TARJETA | $10800.00 | 2026-11-09
  [2] Juan Gómez | PENDIENTE | TARJETA | $19400.00 | 2026-11-09
Id del pedido: 1
Estado actual: PENDIENTE. Nuevo estado:
  1. PENDIENTE
  2. CONFIRMADO
  3. TERMINADO
  4. CANCELADO
Seleccione (Enter conserva): 3
Forma de pago actual: TARJETA. Nueva forma de pago:
  1. TARJETA
  2. TRANSFERENCIA
  3. EFECTIVO
Seleccione (Enter conserva):
[ERROR] Transición inválida: PENDIENTE -> TERMINADO

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Cambiar estado y/o forma de pago
6. Eliminar
0. Volver
Seleccione: 5
  [1] Juan Gómez | PENDIENTE | TARJETA | $10800.00 | 2026-11-09
  [2] Juan Gómez | PENDIENTE | TARJETA | $19400.00 | 2026-11-09
Id del pedido: 2
Estado actual: PENDIENTE. Nuevo estado:
  1. PENDIENTE
  2. CONFIRMADO
  3. TERMINADO
  4. CANCELADO
Seleccione (Enter conserva): 4
Forma de pago actual: TARJETA. Nueva forma de pago:
  1. TARJETA
  2. TRANSFERENCIA
  3. EFECTIVO
Seleccione (Enter conserva):
[OK] Pedido actualizado.

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Cambiar estado y/o forma de pago
6. Eliminar
0. Volver
Seleccione: 5
  [1] Juan Gómez | PENDIENTE | TARJETA | $10800.00 | 2026-11-09
  [2] Juan Gómez | CANCELADO | TARJETA | $19400.00 | 2026-11-09
Id del pedido:
```

Figura 7 – `06_estado_pedido`

Estados del pedido: la transición PENDIENTE → TERMINADO se rechaza; PENDIENTE → CANCELADO es válida.

```
--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
Id de categoría para filtrar (Enter = todas):
  [1] Hamburguesa clásica - $4500.00 - stock 18 - Hamburguesas (disponible)
  [2] Hamburguesa doble cheddar - $6200.00 - stock 15 - Hamburguesas (disponible)
  [3] Pizza muzzarella - $7000.00 - stock 10 - Pizzas (disponible)
  [4] Pizza napolitana - $8200.00 - stock 3 - Pizzas (disponible)
  [5] Gaseosa 500 ml - $1800.00 - stock 39 - Bebidas (disponible)
  [6] Agua mineral - $1200.00 - stock 50 - Bebidas (disponible)
  [7] Flan casero - $2500.00 - stock 12 - Postres (disponible)
  [8] Brownie con helado - $3800.00 - stock 8 - Postres (no disponible)
  [9] Jugo de naranja - $2000.00 - stock 23 - Bebidas (disponible)
```

Figura 8 – `07_stock_devuelto.`

Al cancelar el pedido se devolvió el stock (pizza muzzarella: 10 unidades; gaseosa: 39).

```
=== FOOD STORE – INICIO DE SESIÓN ===
Mail: juan@foodstore.com
Contraseña:

Bienvenido/a, Juan Gómez [USUARIO]

=== FOOD STORE – CLIENTE (Juan Gómez) ===
1. Ver categorías
2. Ver productos disponibles
3. Mis pedidos
4. Mis datos
9. Cerrar sesión
0. Salir
Seleccione: 2
  [1] Hamburguesa clásica - $4500.00 - stock 18 - Hamburguesas (disponible)
  [2] Hamburguesa doble cheddar - $6200.00 - stock 15 - Hamburguesas (disponible)
  [3] Pizza muzzarella - $7000.00 - stock 10 - Pizzas (disponible)
  [4] Pizza napolitana - $8200.00 - stock 3 - Pizzas (disponible)
  [5] Gaseosa 500 ml - $1800.00 - stock 39 - Bebidas (disponible)
  [6] Agua mineral - $1200.00 - stock 50 - Bebidas (disponible)
  [7] Flan casero - $2500.00 - stock 12 - Postres (disponible)
  [9] Jugo de naranja - $2000.00 - stock 23 - Bebidas (disponible)

=== FOOD STORE – CLIENTE (Juan Gómez) ===
1. Ver categorías
2. Ver productos disponibles
3. Mis pedidos
4. Mis datos
9. Cerrar sesión
0. Salir
Seleccione: 3
  [1] Juan Gómez | PENDIENTE | TARJETA | $10800.00 | 2026-11-09
  [2] Juan Gómez | CANCELADO | TARJETA | $19400.00 | 2026-11-09
Id del pedido para ver sus detalles (Enter para volver):

=== FOOD STORE – CLIENTE (Juan Gómez) ===
1. Ver categorías
2. Ver productos disponibles
3. Mis pedidos
4. Mis datos
9. Cerrar sesión
0. Salir
Seleccione: 4
  Nombre:   Juan
  Apellido: Gómez
  Mail:     juan@foodstore.com
  Celular:  2615550002
  Rol:      USUARIO
```

Figura 9 – `08_cliente`

Sesión de cliente: productos disponibles (no se ve el producto no disponible), detalle de sus pedidos y sus datos (sin contraseña).

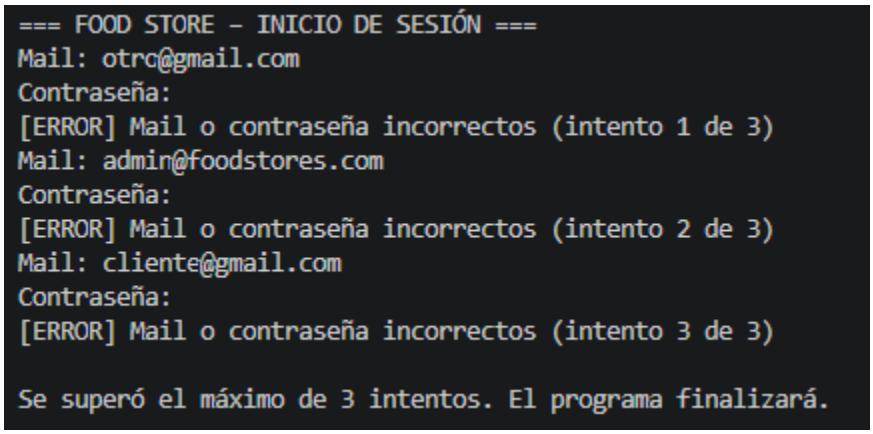


Figura 10 – `09_tres_intentos`

Tres intentos fallidos de inicio de sesión: el programa informa y termina.

3.7 Pruebas

`pruebas.py` ejercita el modelo y los servicios sin pasar por el menú y muestra en pantalla los casos válidos y los errores esperados. Cada línea indica **[OK]** cuando el resultado coincide con lo esperado; en los casos de error se muestra la excepción propia lanzada. La última ejecución verificó **87 de 87** casos, distribuidos así:

Sección	Casos
ENTIDADES: clase abstracta, ids, encapsulamiento, **str** y **eq**	16
PEDIDO: composición, detalles y cálculo del total	17
SERVICIOS: categorías y productos	19
SERVICIOS: usuarios e inicio de sesión	11
SERVICIOS: pedidos, stock y transiciones	20
CONSOLA: menú por rol resuelto con polimorfismo	4
Total	**87**

```
=== PEDIDO: composición, detalles y cálculo del total ===
  [OK] Pedido implementa Calculable
  [OK] arranca PENDIENTE con fecha del día
  [OK] subtotal = cantidad x precio del producto (2 x 1800)
  [OK] total = suma de subtotales (3600 + 1200)
  [OK] mismo producto: se suma la cantidad al detalle existente
  [OK] buscar_detalle_por_producto devuelve None si no está
  [OK] la property detalles devuelve una tupla (no se modifica desde afuera)
  [OK] eliminar_detalle_por_producto recalcula el total
  [OK] cantidad 0 -> CantidadInvalidaError: La cantidad debe ser un entero mayor a 0
  [OK] cantidad negativa -> CantidadInvalidaError: La cantidad debe ser un entero mayor a 0
  [OK] cantidad mayor al stock (4 + 2 > 5) -> StockInsuficienteError: Stock insuficiente de 'Gaseosa': pedido 6, disponible 5
  [OK] eliminar un detalle inexistente -> EntidadNoEncontradaError: El pedido no incluye 'Agua'
  [OK] pedido sin usuario -> DatoInvalidoError: No se puede crear un pedido sin usuario
  [OK] pedido con usuario eliminado -> OperacionNoPermitidaError: No se puede crear un pedido con un usuario eliminado
  [OK] transición PENDIENTE -> CONFIRMADO -> TERMINADO
  [OK] TERMINADO no cambia más -> OperacionNoPermitidaError: Transición inválida: TERMINADO -> CANCELADO
  [OK] PENDIENTE -> TERMINADO no es válida -> OperacionNoPermitidaError: Transición inválida: PENDIENTE -> TERMINADO
```

Figura 11 – `10_pruebas_pedidos`

Fragmento de pruebas.py: pedidos, stock y transiciones (casos válidos y errores esperados).

```
=== CONSOLA: menú por rol resuelto con polimorfismo ===
  [OK] ADMIN recibe MenuAdministrador
  [OK] USUARIO recibe MenuCliente
  [OK] ambos son Menu y se usan con menu.mostrar()
  [OK] Menu es abstracta -> TypeError: Can't instantiate abstract class Menu without an implementation for abstract method 'mostrar'

RESUMEN: 87 de 87 verificaciones correctas.
PS C:\Users\erika\OneDrive\Desktop\Tecnicatura\2do cuatrimestre\Programación II\TPI_FoodStore_Oviedo>
```

Figura 12 – `11_pruebas_resumen`

Resumen final de pruebas.py.

Además, `python -m mypy --strict .` verifica los type hints de los 35 archivos sin errores.

4. Dificultades y soluciones

Dificultad	Solución
Base debía ser abstracta, pero heredar de ABC sin métodos abstractos no impide instanciarla.	Se declaró `__str__` como método abstracto: Base no se puede instanciar y cada entidad queda obligada a tener su representación para los listados.
Al sobrescribir `__eq__` Python elimina `__hash__`, y los objetos se usan en listas y comparaciones.	Se definió `__hash__` coherente con `__eq__` (tipo e id) en Base.
La contraseña no debe poder leerse pero sí validarse.	Se creó una property de solo escritura (sin getter) y el método `verificar_contrasenia()`; leer el atributo lanza AttributeError.

Dificultad	Solución
Importación circular entre Categoria y Producto, que se referencian mutuamente.	Se importó Producto solo para el chequeo de tipos (`TYPE_CHECKING`) y se usaron anotaciones entre comillas; la sincronización la hace el setter de `Producto.categoria`.
Cancelar la creación de un pedido sin tocar el stock cuando falla un detalle.	Se separó el armado (borrador y detalles, que solo validan) del registro, que descuenta el stock y guarda el pedido recién cuando todo es válido.
Ediciones a medias: si el tercer campo editado era inválido, los dos primeros ya habían cambiado.	Los servicios guardan los valores anteriores y los restauran si se produce una excepción; la prueba «una edición inválida no deja cambios parciales» lo verifica.
Ids con huecos cuando un alta fallaba en el constructor.	Los constructores validan antes de llamar a `super().__init__()`. Los pedidos descartados siguen consumiendo su id, como una secuencia de base de datos, y se documentó.
«Solo ADMIN modifica datos» no se cumple solo con ocultar opciones del menú.	Los servicios reciben el usuario que actúa y exigen rol ADMIN; las pruebas intentan operaciones de cliente y verifican el rechazo.
El menú no debía caerse nunca, pero capturar Exception también captura EOFError y con la entrada cerrada el programa entraba en un bucle infinito.	`ejecutar_seguro` vuelve a lanzar EOFError y `main()` lo captura junto con KeyboardInterrupt para terminar con un mensaje.
Mails y nombres con distinta capitalización (Juan@... vs juan@...).	La comparación se hace en minúsculas para el mail y el nombre de categoría; el mail de un usuario eliminado sigue reservado.
Capturas y flujos repetibles sin escribir cada entrada a mano.	Se simularon las entradas del teclado (`input` y `getpass`) con un script auxiliar y se ejecutó cada sesión en un proceso nuevo, para que los ids arranquen de cero.

5. Conclusiones

El proyecto cumple las historias de usuario del backlog y los requisitos del modelo: clase abstracta, interfaz, herencia y sobrescritura, encapsulamiento con validaciones, composición y agregación, colecciones, excepciones propias y polimorfismo para los menús por rol. Separar el código en entidades, servicios y consola facilitó probar las reglas de negocio sin menú y confirmó que cada capa tiene una responsabilidad clara.

Como mejoras futuras se podrían guardar las contraseñas con hash y sal, persistir los datos en archivos o en una base de datos (los servicios ya aíslan la colección, por lo que solo habría que reemplazar su almacenamiento) y permitir que el cliente cree sus propios pedidos.

6. Bibliografía y webgrafía

- Python Software Foundation. *The Python Tutorial – Classes*. docs.python.org/3/tutorial/classes.html
- Python Software Foundation. *The Python Tutorial – Errors and Exceptions*. docs.python.org/3/tutorial/errors.html
- Python Software Foundation. *The Python Language Reference – Data model*. docs.python.org/3/reference/datamodel.html
- Python Software Foundation. *abc – Abstract Base Classes*. docs.python.org/3/library/abc.html
- Python Software Foundation. *enum – Support for enumerations*. docs.python.org/3/library/enum.html
- Python Software Foundation. *datetime – Basic date and time types*. docs.python.org/3/library/datetime.html
- Python Software Foundation. *getpass – Portable password input*. docs.python.org/3/library/getpass.html
- Python Software Foundation. *typing – Support for type hints*. docs.python.org/3/library/typing.html
- van Rossum, G., Lehtosalo, J., Langa, Ł. *PEP 484 – Type Hints*. peps.python.org/pep-0484
- Talin, van Rossum, G. *PEP 3119 – Introducing Abstract Base Classes*. peps.python.org/pep-3119
- van Rossum, G., Warsaw, B., Coghlan, N. *PEP 8 – Style Guide for Python Code*. peps.python.org/pep-0008
- The mypy project. *mypy documentation*. mypy.readthedocs.io
- Universidad Tecnológica Nacional. *Trabajo Práctico Integrador – Programación II: Food Store*. Enunciado de la cátedra (Tecnicatura Universitaria en Programación a Distancia).

7. Enlaces

Repositorio GitHub

Video demostración (10 a 15 minutos, con permisos públicos de visualización): [Lo entregaré en la segunda entrega con las correcciones.]